
從第一原理理解 Ray

從一次 `remote()` 呼叫，到第一次讀懂 Ray Issue

Version 1.0 · Short Main Path

Prototype for Review

Status	Prototype for Review
Technical baseline	Ray 2.58.0
Source tag	ray-2.58.0
Language	zh-TW
Author	Yen-Hua Chen
License	CC BY 4.0
建議閱讀時間	2–3 小時

Original source:

<https://github.com/hikaru-212/open-source-from-first-principles>

這是一份教學原型，不是完整的 Ray 規格或 API 手冊。若本文與官方資料有差異，應以 Ray 2.58.0 官方文件，以及 `ray-2.58.0` 標籤下的原始碼與測試為準。

`master` 只能作為開發中的證據，不能代表 Ray 2.58.0；GitHub issue 與尚未合併的 PR 也不是公開契約。

Ray 到底是什麼？

Ray 是分散式執行 runtime / framework，讓程式把 task 與有狀態的 actor 分散到多台機器的 CPU / GPU 上執行。

使用者宣告要執行的計算與資源需求，Ray 負責選擇在哪裡、由哪個 worker 執行，配置邏輯資源、追蹤結果，並管理 object 的搬移與生命週期；worker / node failure 後，依各自政策與適用條件重試或重建。它解決的是分散執行的協調問題，恢復不代表從中斷處接續。

Ray 不負責 application-level exactly-once external effects、持久化的業務流程歷史，或 database transaction semantics。

有了這個定位，第一章再問：`f.remote()` 之後，到底存在什麼？

接下來，這份指南反覆追問：

現在實際存在什麼？誰知道什麼？哪個物理狀態消失了？誰有能力恢復它？

怎麼讀這份指南

第一次閱讀時，不需要記住所有 Ray internal 名稱。每章先只回答三個問題：

1. 現在存在的 logical state 是什麼？
2. 哪個 physical component 正在做事？
3. failure 發生後，哪些東西還在，哪些消失？

Deep Dive、source file 與 issue number 都可以留到第二遍再看。

第一章：一次 `remote()` 呼叫之後，到底產生了什麼？

先看最普通的 Ray 呼叫：

```
@ray.remote
def f(x):
    return x * 2

ref = f.remote(21)
```

`f.remote(21)` 回傳時，函式已經執行了嗎？某個 worker 已經接到工作了嗎？`ref` 是否證明答案已經算出來？

都不一定。

先記住這一句
Task 是工作的 logical identity，不是 process。

此刻可以確定的是：這次 `.remote()` 已建立並提交一份可追蹤的 logical task，並回傳對未來結果的 `ObjectRef`。`ObjectRef` 表示「結果可經由這個 reference 取得」，不表示 user code 已經執行。

最常見的錯誤直覺，是把 task 當成一個固定 process：

錯誤直覺：

```
一個 remote call
  ↓
一個固定 worker
  ↓
一份結果
```

實際上需要分成三層：

```
f.remote()
  ↓
logical task
  ↓
attempt 0 — worker W1
  |
  └─ failure
      ↓
    attempt 1 — worker W2
```

task 是邏輯身份；attempt 是一次實際執行嘗試；worker 是執行該 attempt 的 process。在允許 retry 的故障下，同一個 task 可以產生新的 attempt，而且不必由原 worker 執行。

反過來，一個普通 task worker 通常也能依序執行許多互不相關的 task。因此：

logical work 不等於 physical worker。

actor 同樣需要區分邏輯身份與物理 process。actor 是具有身份和記憶體狀態的抽象；實際承載它的是專用 actor worker。

如果 actor 被允許 restart，Ray 可以保留同一個邏輯 actor 身份，建立新的 worker process，並重新執行 constructor：

```
Actor A
├─ incarnation 1: worker W1、heap H1
└─ restart
    ↓
  incarnation 2: worker W2、全新的 heap H2
```

保留下來的是 actor 身份與 handle 的指向關係，不是舊 process 的 stack、heap 或 application state。

Ray 保證的邊界：一次 `.remote()` 會建立可追蹤的 logical task 與結果 reference；retry 可以為同一 task 建立新的 attempt。

不能由此推論：worker 已選定、user code 已開始、結果已存在，或外部副作用尚未發生。

Checkpoint

`f.remote()` 已回傳 `ObjectRef`，但你還沒有呼叫 `ray.get()`。此刻哪些邏輯狀態已經存在？哪些關於 worker、user code 與結果的事仍然未知？

Deep Dive (選讀)

- `TaskID`、attempt number、`WorkerID` 與 `ActorID`
- actor incarnation 與 intended-recipient worker 檢查

第二章：從資源需求到真正執行，中間還有多少步？

假設 Ray 已經收到一個 logical task。下一個常見直覺是：

有資源就執行；顯示 `scheduled` 就表示正在跑。

這把多個不同的物理轉換壓成了一個詞。

先記住這一句

Scheduled 不等於 executing。

你送出一個需要 8 CPU 的 task，但它一直 pending。這可能代表 cluster 根本沒有任何 node 能提供 8 CPU，也可能有 node 能提供，只是資源目前被其他工作占用。看起來都叫 pending，但系統下一步能做的事完全不同。

一個 task 從 submission 到結果可被觀察，大致經過：

```
submission
  ↓
dependency readiness
  ↓
resource feasibility
  ↓
worker lease + logical resource allocation
  ↓
dispatch
  ↓
user-code execution
  ↓
result publication
```

Submission 表示 Ray 收到 task specification。這時 `ObjectRef` 可以已經回傳，但工作尚未執行。

如果參數包含其他 `ObjectRef`，Ray 還要等待相依資料可取得。這是 **dependency readiness**，不是 CPU scheduling。

接著要區分兩個容易混淆的問題：

- **Feasible**：是否存在資源形狀足以執行它的 node？
- **Currently available**：那些 logical resources 現在是否空閒？

例如，一個要求 8 CPU 的 task，在所有 node 都只有 4 CPU 時是 infeasible；如果某個 node 有 8 CPU，但目前被占用，則是 feasible but unavailable。兩者都可能 pending，物理原因卻不同。

當 raylet 找到位置，Ray 會配置 logical resources 並授予 worker lease。提交端取得 worker 後，仍要 dispatch task；worker 收到工作後，也要解析 reference、反序列化函式與參數，才能進入 user code。

所以 successful scheduling 最多證明：

Ray 在那個時間點找到可行位置、配置 logical resources，並選出預定執行的 worker。

它不證明 dispatch 已送達、user code 已開始、執行已完成，或結果已傳回 caller。

Logical CPU 不是 CPU pinning

Ray 的 `num_cpus=1` 是 admission control。它限制 Ray 同時配置多少邏輯資源，但不代表：

- process 被固定在某一顆實體 CPU；
- 作業系統只允許它使用一顆 CPU；
- user code 不能建立額外 threads；
- worker 之間獲得硬體層級隔離。

Placement group：reservation 不等於 execution

placement group 是進階的 reservation 例子：它把多個 logical resource bundles 一起配置。初始建立採 gang / atomic reservation；所有 bundles 都能保留才成功，否則不保留任何 bundle。

`pg.ready()` 因而證明初始 reservation 已完成，但不證明 actor 已初始化、task 已開始，或應用程式已產生有用結果。

Ray 保證的邊界：logical resources 控制 Ray 接受與配置工作的方式；初始 placement-group 建立提供 bundles 的原子 reservation。

不能由此推論：物理 CPU 隔離、user code 已進入，或 reservation 已產生執行結果。

Checkpoint

某個 task 已取得 worker lease，Ray 也已配置所需 logical CPU，但 worker 還在處理 dispatch 與參數反序列化。此時「scheduled」證明了什麼？又還不能證明什麼？

Deep Dive (選讀)

- placement-group bundle scheduling 與 failure recovery
- spillback
- GPU visibility
- blocking call 與 logical CPU accounting

第三章：誰擁有結果？結果的 bytes 又在哪裡？

考慮以下情境：

Driver 建立 task，另一台機器上的 worker 執行它，結果 bytes 最後存到第三個位置。那麼，這個結果到底「算誰的」？

先記住這一句

Owner、executor 與 byte storage location 是三件不同的事。

```
driver D
  | 建立 task、收到 ObjectRef
  ▼
worker W
  | 執行函式
  ▼
node N 的 object store
  儲存結果 bytes
```

哪個角色「擁有」這個 object？

直覺可能回答：儲存 bytes 的 node。但 Ray 的 ownership 不是「資料放在哪裡」。

對一般 remote task 而言，提交工作並建立原始 `ObjectRef` 的 worker 是 owner。實際執行 user code 的 worker 是 executor；結果 bytes 則可能存在某個 worker 的記憶體、某個 node 的 object store，或多個 replicas 中。

因此必須固定這個區分：

```
owner
  != executor
  != byte storage location
```

owner 維護 logical object 的生命週期資訊；executor 知道自己執行了什麼；object storage 保存 bytes。這些角色可能偶爾位於同一個 process 或 node，責任卻不同。

ObjectRef 與生命週期

只要可被 Ray 追蹤的 `ObjectRef` 仍存在，distributed reference tracking 就能讓系統知道物件仍被使用。reference 也可能出現在 pending task 的參數或其他 Ray object 中。

初學者暫時不需要完整的 reference-counting protocol，只要記得：

`ObjectRef` 不只是地址；它也參與 Ray 對物件生命週期的判斷。

Bytes 消失，不一定等於 logical object 消失

如果一份 object replica 隨 node 消失，Ray 會先尋找其他 copy。若沒有 copy，而結果原本由可重試的 task 產生，Ray 有時可以利用 lineage 重新執行 producer task：

```
object bytes lost
  ↓
another replica?
├─ yes → 使用 replica
└─ no
  ↓
producer lineage usable?
├─ yes → 重新執行 producer
└─ no → object loss
```

這不是從備份還原 bytes，而是再次執行產生 bytes 的工作。因此 producer 的外部副作用也可能再次發生。

`ray.put()` 不具備相同的 producer-task reconstruction 路徑：它放入的是既有值，沒有一個原始 producer task 可以自然重跑。

普通 task 有自己的 retry / reconstruction eligibility。actor method 預設 `max_task_retries=0`，因此在這個預設下，由 actor task 產生的 object 不是 reconstruction candidate；設定非零的 actor-method retry policy，才會改變這項 eligibility。

Owner 消失是不同的失敗

如果只是 byte replica 消失，但 owner、其他 replicas 或 producer lineage 還在，Ray 可能恢復物件。

如果 owner 死亡，logical object 的 metadata 與 reconstruction authority 也會失去。即使某些 bytes 曾存在其他 node，也不能因此推論 ownership 仍完整存在。

Ray 保證的邊界：owner 協調 logical object；bytes 可以位於不同位置；符合條件的 task-produced object 可以透過 lineage reconstruction。

不能由此推論：存放 bytes 的 node 就是 owner、任何 object 都能 reconstruction，或 owner death 只是少了一份 replica。

Checkpoint

如果 object 的唯一 byte replica 消失，但 owner 與 producer lineage 都還在，Ray 可能重建什麼？如果改成 bytes 仍在、但 owner 死亡，為什麼結果不同？

Deep Dive (選讀)

- nested references
- out-of-band `ObjectRef` serialization
- lineage retention 與 eviction
- detached actors 與 detached placement groups

第四章：Process 或 node 消失時，真正失去的是什麼？

「機器掛了，Ray 會恢復」不是足夠精確的模型。一台 node 上可能同時存在 task worker、actor worker、object bytes 與 placement-group bundles；它們分別由不同機制處理。

先記住這一句

「node 掛掉」不是一種 failure，而是多種 state 同時消失。

想像一台 node 同時跑著一個普通 task、一個 actor，並存放一份 object replica 與一個 placement-group bundle。現在整台 node 消失。問題不是「Ray 會不會 retry」，而是這四種東西分別失去了什麼。

失敗	立即消失的物理狀態	可能保留的邏輯身份	誰可能發起恢復	無法自動恢復
task worker death	該 attempt 的 stack、heap	<code>TaskID</code> 、結果 refs	task retry 路徑	process-local state、外部效果
actor worker death	actor heap、active call stack	<code>ActorID</code> ，若允許 restart	actor lifecycle	未自行保存的 application state
node death	node 上的 worker 狀態與不可用 replicas	依各 entity policy 而定	task、actor、object、PG 各自處理	整台 node 的一致快照
owner death	ownership metadata 與 recovery authority	detached entity 可能獨立存活	通常進入終止與 cleanup	owned object 的 reconstruction authority

Worker death：失去的是一次 attempt

task worker 在 user code 中 crash，舊 attempt 的 stack 與 heap 立即消失。如果 retry budget 尚未用完，Ray 可以為同一 logical task 建立新的 attempt。

但舊 attempt 已完成的外部寫入，不會因 process 消失而撤銷。

Actor worker death : restart 不等於 state restoration

actor 若允許 restart，Ray 可以建立新 process，並重新執行 constructor。舊 actor heap 不會被搬到新 worker。

```
actor worker dies
  ↓
old heap disappears
  ↓
new process starts
  ↓
constructor runs again
```

application state 只有在程式自行從 durable storage 或 checkpoint 載入時，才可能恢復。

Node death : 多條 recovery path 同時發生

node failure 不會觸發一個包辦所有事情的「node rollback」。它可能同時造成：

- task attempt 進入 task retry；
- actor 依 restart policy 決定是否重建；
- object 嘗試使用 replica 或 reconstruction；
- lost placement-group bundles 等待重新配置。

這些政策彼此獨立。placement group 的 surviving bundles 可以繼續保留資源，而 lost bundles 另行恢復；初始 gang reservation 不代表故障後只能整組成功或整組失敗。

Owner death : 終止本身也是分散式轉換

Owner death 不只是某個 process 消失。它還可能觸發 resources、references、worker leases 與 pending work 的分散式 cleanup。

不同元件可能參與 cleanup，因此「到達 terminal state」本身也是一次需要收斂的 distributed transition，而不是一個瞬間完成的本地動作。

Ray 保證的邊界：不同 logical entity 依各自 policy 建立新的 attempt、process、object producer 或 bundle placement。

不能由此推論：舊 process state 被還原、整台 node 被一致回復，或所有 entity 共用一套 recovery policy。

Checkpoint

一台 node 同時承載 task T、actor A、object O 的唯一 replica，以及 placement-group bundle B。node 消失後，為什麼不能只問「Ray 會不會 retry」？你需要分別確認哪些 recovery policy？

Deep Dive (選讀)

- GCS 與 head-node recovery
- placement-group partial recovery
- owner-death cleanup convergence : worker lease 、 resource accounting 、 pinned arguments 與 dependency bookkeeping
- Ray #64627 : cleanup concern 有 implementation evidence , 但完整 production leak 仍 unresolved

第五章：Recovery 不是時間倒流，而是新的執行

這是理解 Ray fault tolerance 的核心。

先記住這一句

Recovery 是新的 execution，不是時間倒流。

Ray 有數種外觀看似「再試一次」的機制，但它們的 trigger、保留身份與重新執行內容不同：

機制	典型 trigger	保留的邏輯身份	實際重新發生的事
task retry	task worker 或 node failure	同一 logical task 與結果 refs	建立新的 task attempt
actor restart	actor worker 或 node failure	同一 ActorID	建立新 process，重跑 constructor
actor-method retry	actor unavailable、死亡或重啟	同一 logical actor task	method code 可能再次執行
object reconstruction	object bytes 無可用 copy	同一 logical object	producer task 可能再次執行

Task retry：新的物理 attempt

task retry 保留 logical task 身份，但不保留 worker process 的執行現場。

如果 attempt 0 在送出網路請求後 crash，attempt 1 不會自動知道外部服務是否已接受該請求。這需要應用程式自己的 idempotency 或確認機制。

Actor restart：重建 process

`max_restarts` 控制 actor process 是否可以重建。restart 會重新執行 constructor，因此 constructor 的外部副作用也可能重複。

它不會自動恢復舊 heap、回到失敗 method 的前一行，或還原 method 執行前的 actor state。失敗 method 是否再執行，則由 actor-method retry policy 另外決定。

Actor–method retry : method 可能再次執行

即使沒有 method retry，caller 看到 error 也不一定能證明 method 沒有執行：

```
actor method 寫入外部資料庫
    ↓
method 已執行成功
    ↓
actor 在 completion 傳回前死亡
    ↓
caller 收到 failure
```

如果開啟 method retry，新的 attempt 可能再寫一次。Ray 能重新執行計算，但不會替外部資料庫建立 exactly–once transaction。

Object reconstruction : 重跑 producer

object reconstruction 可能重新執行產生結果的 task，而不是直接還原原 bytes：

```
producer attempt 0
├─ external effect
└─ object bytes — lost
    ↓
producer attempt 1
├─ external effect again?
└─ new bytes
```

若 producer 含有非幂等外部效果，application 必須自行處理重複。

因此應固定這個邊界：

```
recovery
  != continuation
  != rollback
  != exactly–once external effect
```

caller–visible error 是一項觀察結果，不是完整執行歷史。正確問題不是「Ray 是否只 call 一次」，而是：

哪個 logical operation 正在 retry？哪個 physical attempt 可能已進入 user code？外部系統如何辨認重複？

Ray 保證的邊界：各 recovery mechanism 依自己的 policy 建立新的執行或 process，並保留特定 logical identity。

不能由此推論：process continuation、application rollback、外部效果撤銷，或 failure 證明 user code 從未執行。

Checkpoint

actor method 已把訂單寫入外部資料庫，但 actor 在成功回覆送達前死亡。caller 收到 error。為什麼這不能證明寫入沒有發生？如果 method retry 開啟，application 還需要哪一類保護？

Deep Dive (選讀)

- Ray #44719：effective retry budget 與 restart-time policy 是兩層問題；issue 提案不是現行契約
- actor ordering under retries
- application idempotency、operation ID 與 fencing

第六章：從 Ray User 到 Ray Contributor

Contribution Ready 不代表讀完整個 Ray repository，而是能把一個有限行為沿著證據追到負責它的 implementation。

先記住這一句

Contribution Ready 是能追一條 evidence chain，不是看懂整個 Ray。

建議使用同一條調查路徑：

```
public behavior
  ↓
pinned stable documentation
  ↓
focused test
  ↓
owning implementation
  ↓
failure timeline
  ↓
issue diagnosis / minimal fix
```

先固定版本

調查前先確認使用者安裝的 Ray release、對應 Git tag，以及文件是否描述同一版本。

本指南固定 Ray 2.58.0 與 `ray-2.58.0`。master 可以顯示開發方向，不能反過來定義 stable 行為；issue comment 也只能提供假說，不能取代公開文件與 stable evidence。

主路徑：task retry 與 object reconstruction

假設使用者報告：

worker 死亡後 task 重新執行；稍後 object bytes 遺失時，producer 又跑了一次。

先拆成兩個公開問題：

1. task worker failure 何時觸發 retry？
2. object loss 何時觸發 producer reconstruction？

接著依序查證：

1. 閱讀 Ray 2.58 的 task/object fault-tolerance 文件，確認 retry budget、owner、replica 與 lineage 的條件。
2. 閱讀 `test_reconstruction.py` 中的 focused test，觀察 task-produced object 如何 reconstruction，以及它與 `ray.put()` 的差別。
3. 接著進入 `src/ray/core_worker/` 下的 C++ `core_worker` 層，從 `TaskManager` 與 `ObjectRecoveryManager` 找到 retry accounting 與 recovery decision。
4. 畫出 failure timeline，確認每個 transition 由誰推動、使用什麼狀態、失敗時 caller 看見什麼。

```
ObjectRef still live
  ↓
bytes unavailable
  ↓
another replica?
  ↓
lineage eligible?
  ↓
producer retry or terminal error
```

focused test 能證明 stable repository 明確保護某個案例，但不代表所有相鄰情境都有同樣保證。source 能顯示 implementation 如何運作，也不表示每個細節都是 public contract。

State API 則是診斷證據。它的結果可能因資料來源不可用、查詢上限或 garbage collection 而不完整，也不保證是全域一致的瞬間快照。因此，查不到 task 不等於 task 從未存在；顯示 `RUNNING` 也不能證明外部副作用的狀態。

即使最後沒有 patch，調查仍可能產生貢獻：縮小 reproduction、補 focused test、澄清文件邊界，或找出兩個被混在一起的 state machines。

Contribution Ready 不代表知道 Ray 的全部。

它是能把一個 bounded behavior，從公開契約追到證據，再追到負責那次 state transition 的 implementation。

這條調查路徑能建立的證據邊界：stable 文件、測試與 source 能為特定版本建立可檢查的 evidence chain。

不能由此推論：master 等於 stable、issue comment 等於 contract，或一筆 State API 記錄代表完整 runtime truth。

Checkpoint

你收到一個「node failure 後 producer 意外重跑兩次」的 issue。在修改 source 前，應如何固定版本、確認公開契約、找到 focused test、區分 task retry 與 object reconstruction，並判斷 State API 輸出能證明到哪裡？

Deep Dive (選讀)

- actor restart → actor failure test → `ActorTaskSubmitter` / `ActorManager`
- placement-group failover → focused failover test → GCS placement-group manager ;
#65147 的 2.58 reproduction 尚未獨立確認，PR #65970 尚未合併
- #44719：已確認部分 policy layering，但其提案不是現行契約
- #64627：cleanup path concern 有 source evidence，完整 production leak 仍 unresolved

各章權威資料

以下 `docs.ray.io/en/latest` 文件在本原型查證時對應 Ray 2.58.0，但網址本身會隨 stable release 更新。固定 implementation evidence 應以 `ray-2.58.0` tag 為準。

第一章

- `Ray Tasks`
- `Ray Actors`
- `common.proto` — task attempt identity

第二章

- `Resources`
- `Placement Groups`
- `task-lifecycle.rst`

第三章

- `Object Fault Tolerance`
- `ObjectRef Reference Counting`
- `object_recovery_manager.cc`

第四章

- Task Fault Tolerance
- Actor Fault Tolerance
- Node Fault Tolerance
- `test_placement_group_failover.py`

第五章

- Task Fault Tolerance
- Actor Fault Tolerance
- `task_manager.cc`
- `actor_task_submitter.cc`

第六章

- `test_reconstruction.py`
- `TaskManager`
- `ObjectRecoveryManager`
- Ray State API

Case studies：不是 Ray 2.58 公開契約

- Ray #44719：部分結構分析已獲 source 支持；提案語義不是現行契約
- Ray #65147：結構性 concern 仍相關；尚未獨立重現於 Ray 2.58
- Ray PR #65970：尚未合併
- Ray #64627：cleanup concern 有 implementation evidence；完整 production leak 仍 unresolved

方法來源與查證聲明

本指南的第一原理教學方法，源自 Yen-Hua Chen 在 *Streaming System + Compass* 中記錄的工程推理：先辨認 physical actors、狀態轉換、證據邊界與 failure timeline，再討論抽象與 API。

- Streaming System + Compass — Yen-Hua Chen
- <https://github.com/hikaru-212/streaming-system-compass>
- Documentation license: CC BY 4.0

Compass architecture 並未直接套用到 Ray。Ray-specific claims 已獨立對照 Ray 2.58.0 文件、tagged source 與 focused stable tests；物理機制不相符的類比則明確拒絕：

- Ray actor != Compass semantic authority
- lineage reconstruction != event-log replay

- GCS != application source of truth
 - placement-group reservation != distributed transaction
 - actor reincarnation != ownership transfer
-

作者備註

本 short main path 刻意省略 Ray 產品導覽、完整 GCS / raylet / scheduler internals、reference-counting edge cases、完整 placement-group recovery，以及尚未由 stable evidence 支持的 issue 結論。

未來 Expanded / Deep Dive 版可再處理 actor retry ordering、placement-group partial recovery、owner-death cleanup convergence、application fencing，以及從 reproduction 到 regression test 與 minimal patch 的完整貢獻流程。